20XC87 - Mobile Security Lab

Project Aeterna: The Silas Vane Incident

22PC20 - N. Muthu Meenakshi

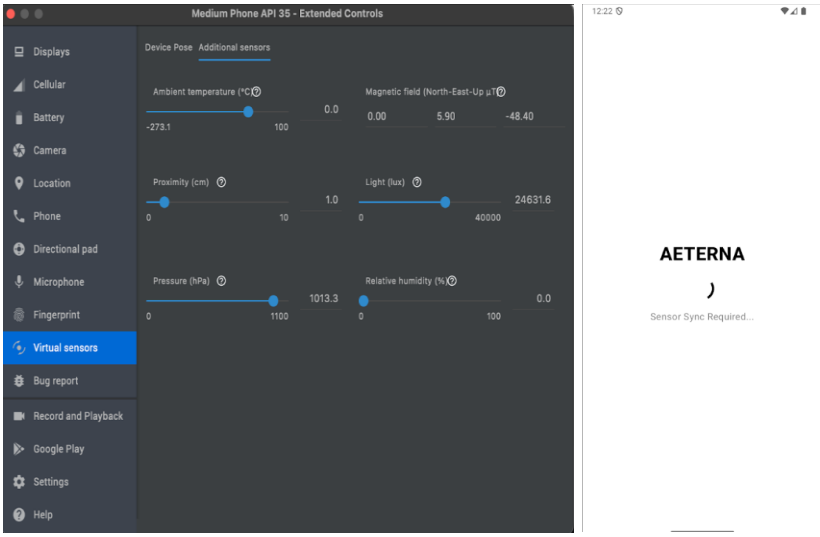
22PC38 – Vishnu Priya M

Walkthrough:

Flag 1: The Biometric Light Trap

The application initializes in a "Sync" state, blocking all user input. The code monitors the Sensor.TYPE_LIGHT hardware. To bypass this, a player must realize the app is looking for a lux value of exactly 0.0. This simulates a physical "darkroom" requirement.

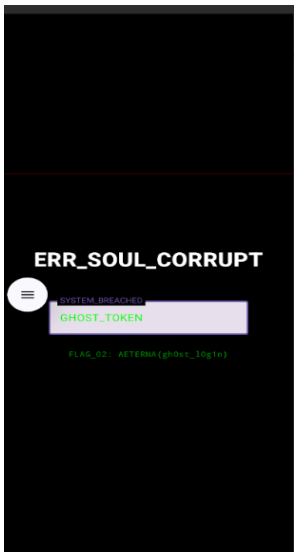
- **Breach:** Use the Emulator's Extended Controls to manually slide the Light Sensor to 0.
- **The Result:** The isDarknessAchieved state triggers, revealing the credential input field.



Flag 2: The Algebraic Gate

Instead of a standard password, the app uses a mathematical constraint. In the code, the player sees an integer array called gate. The function solveAlgebraicGate checks if input[i].code + (i * 7) == gate[i]. A search for "GHOST_TOKEN" in the source code will return zero results.

- **Breach:** The player must reverse the algebra: $Char = Gate[i] - (i \times 7)$.
- **The Result:** Entering **GHOST_TOKEN** triggers the "Glitch UI" and displays **AETERNA{gh0st_l0g1n}**.



Flag 3: The Manifest Permission Ghost

This flag is hidden outside the Java/Kotlin code entirely. It is placed within the AndroidManifest.xml as a custom <permission> tag. To further hide it, the flag value is Base64 encoded.

- **Breach:** Decompile the APK and inspect the Manifest for com.aeterna.soul.ACCESS_QUVURVJOQXttNG4xZjNzdF92MXMxMG59.
- **Result:** Decoding the string reveals **AETERNA{m4n1f3st_v1s10n}**.

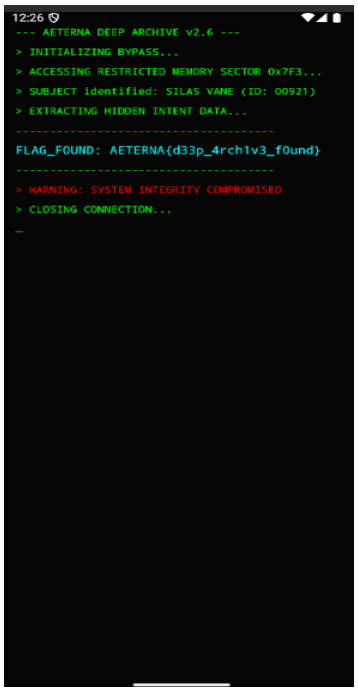
```
<permission
    android:name="com.aeterna.soul.ACCESS_QUVURVJOQXttNG4xZjNzdF92MXMxMG59"
    android:protectionLevel="normal" />
```

Flag 4: The Deep Link Intent Bypass

The DeepArchiveActivity is marked as exported="true", but there is no button in the UI to reach it. The Manifest reveals an intent-filter with a custom scheme: aeterna2026://recovery.

- **Breach:** Use ADB to force the Android system to launch the hidden activity.
- **Command:** adb shell am start -a android.intent.action.VIEW -d "aeterna2026://recovery"
- **Result:** The terminal-style screen reveals **AETERNA{d33p_4rch1v3_f0und}**.

```
vishnupriya@Vishnus-MacBook-Pro Aeterna % adb shell am start -a android.intent.action.VIEW -d "aeterna2026://recovery"
Starting: Intent { act=android.intent.action.VIEW dat=aeterna2026://recovery/... }
```



Flag 5: The JNI Binary Vault

The "Master Key" is locked inside a compiled C++ library (libsoul-engine.so). When the glitched title is clicked, it calls getVaultSecret(0). The C++ logic only returns the flag if the input is 0xDEADBEEF. Since the Kotlin code is hardcoded to send 0, the UI always shows an error.

- **Breach:** Use Frida to hook the native function and inject the correct hex code into the process memory.
- **Result:** The UI updates to show the decrypted master key: **AETERNA{b1n4ry_sl3uth}**.

```
extern "C" JNIEXPORT jstring JNICALL
Java_com_aeterna_soul_archive_MainActivity_getVaultSecret(
        JNIEnv* env,
        jobject MainActivity /* this */,
        jint access_code) { // Accepts an integer now

    // The "Lock": 0xDEADBEEF (3735928559 in decimal)
    if (access_code != 0xDEADBEEF) {
        return env->NewStringUTF( bytes: "ERROR: INVALID_VAULT_KEY");
    }

    // Flag 5: n4t1v3_s0ul_r3v34l
    unsigned char secret[20] = {
            [0]: 0x2E,  [1]: 0x76,  [2]: 0x36,  [3]: 0x73,  [4]: 0x71,  [5]: 0x31,  [6]: 0x1D,  [7]: 0x31,  [8]: 0x72,  [9]: 0x77,
    };

    for(int i = 0; i < 18; i++) {
        secret[i] = secret[i] ^ 0x42;
    }
```

Flag 6: The Runtime Forensic Leak

When the app starts, it silently writes a hidden file to the internal filesystem: .silas_memory. The flag inside this file is "char-shifted" in the source code (e.g., 'A' is stored as 'B') to prevent strings-based detection.

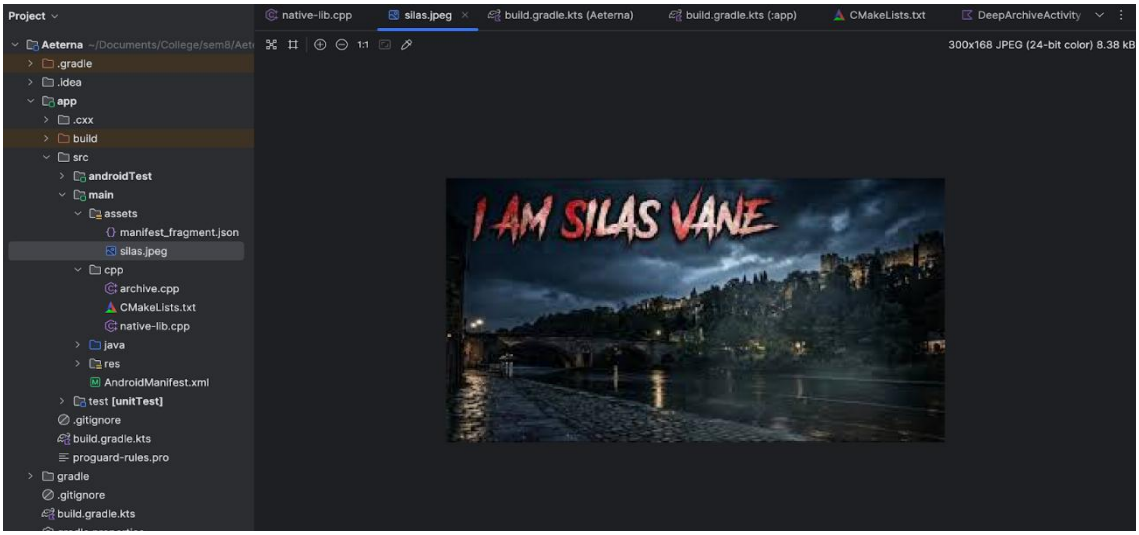
- **Breach:** Use adb shell and run-as to bypass Android's folder permissions and read the hidden file.
- **Command:** cat /data/data/com.aeterna.soul.archive/files/.silas_memory
- **Result:** Reveals **AETERNA{f0r3ns1cs_m4st3r_v4n3}** and the hint for the JNI Vault key.

```
vishnupriya@Vishnus-MacBook-Pro ~ % adb shell
emu64a:/ $ run-as com.aeterna.soul.archive
emu64a:/data/user/0/com.aeterna.soul.archive $ cat files/.silas_memory
AETERNA{f0r3ns1cs_m4st3r_v4n3}
TRACE: 0xDEDABEFEemu64a:/data/user/0/com.aeterna.soul.archive $
```

Flag 7: The Steganographic Image Asset

This flag involves "Data Append" steganography. The flag is not in the code or the database; it is appended to the end of a physical image asset (e.g., spooky_avatar.png) located in the assets/ folder.

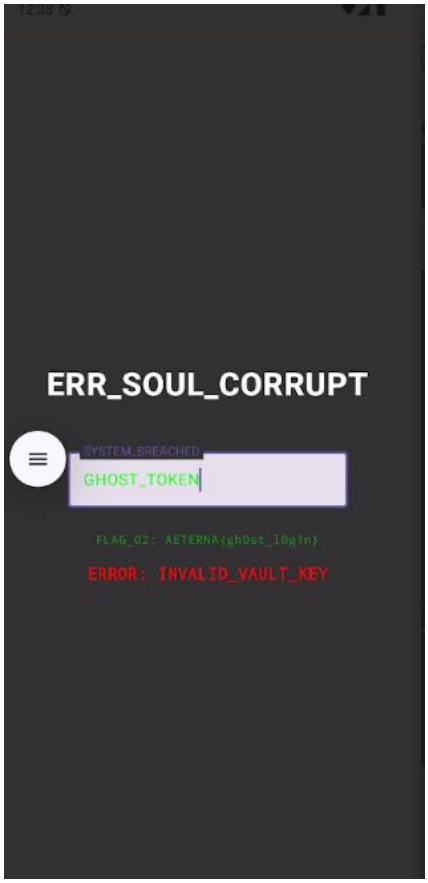
- **Breach:** Use the Linux strings utility or a Hex Editor to look at the "tail" of the image file.
- **Result:** The player finds the trailing string: **AETERNA{v1su4l_st3g0_m4st3r}**.




```
vishnupriya@Vishnus-MacBook-Pro Aeterna % strings app/src/main/assets/silas.jpeg | grep AETERNA
AETERNA{v1su4l_st3g0_m4st3r}
```

Flag 8: The Developer Mode XML

The final flag is a "Behavioral Trigger." It requires the player to find a hidden tap-counter in the code. Tapping the "ERR_SOUL_CORRUPT" title exactly 7 times triggers a secret function. This function "decrypts" a garbled string (Bfufsob...) and writes it into the Android SharedPreferences system under the file vane_logs.xml.

This challenge is particularly difficult because there is no visual feedback when the taps occur. The player must rely entirely on their analysis of the decompiled Kotlin logic. Once the 7 taps are completed, they must perform a second forensic search using ADB to locate the newly generated XML file. Executing cat shared_prefs/vane_logs.xml reveals the final flag: **AETERNA{d3v_m0d3_u/l3ck3d}.**

```
vishnupriya@Vishnus-MacBook-Pro Aeterna % adb shell run-as com.aeterna.soul.archive cat shared_prefs/vane_logs.xml
<?xml version='1.0' encoding='utf-8' standalone='yes' ?>
<map>
    <string name="build_id">Aeterna{d3v_m0d3_u/l3ck3d}</string>
</map>
```